

PagedAttention 技术调研报告

PagedAttention 技术调研报告

基于论文 (SOSP 2023)、vLLM 代码库 (83,139 nodes / 490,393 edges)、codebase-memory 深度分析 2026-06-29 | KG System

目录

- 论文核心设计
- 整体推理流程
- PagedAttention 核心算法
- Block 内存管理机制
- 调度器设计
- 代码实现分析
- 关联生态

1. 论文核心设计

1.1 问题定义

LLM 推理时显存分布（以 13B 模型在 NVIDIA A100 40GB 为例）：

显存占用	比例	特点
模型权重 (Parameters)	65%	静态常量，推理全程不变
KV Cache	30%	动态增长/收缩，每请求独立
激活值 (Activations)	~5%	临时张量，用完即释放

核心矛盾：现有推理系统（FasterTransformer, Orca）要求 KV cache 存储在连续内存空间中，但 KV cache 具有三个独特性质：- 动态增长（每生成一个 token 就变大）- 寿命不可预知（不知道请求何时结束）- 长度不可预知（不知道会生成多长）

导致：- **内部碎片：**预分配请求最大长度 (2048 tokens)，实际平均只用 ~200 tokens，浪费 90% - **外部碎片：**不同请求预分配不同大小，造成内存空洞 - **无共享：**beam search、parallel sampling 产生的多个序列无法共享 KV cache

实测数据：现有系统仅 **20.4% ~ 38.2%** 的 KV cache 内存被有效利用。

1.2 PagedAttention 核心思想

受操作系统虚拟内存分页机制启发，将 KV cache 划分为固定大小的 **Block**：

操作系统虚拟内存 → PagedAttention

物理页框 (page frame)	→	KV cache block
虚拟地址 (virtual addr)	→	逻辑 token 位置
进程 (process)	→	请求 (request)
页表 (page table)	→	block table

四个核心设计原则：

- 1. 分块管理 (Block-level Allocation)**
 - KV cache 按固定大小 block 分配（典型 block_size=16/32 tokens）
 - 不再要求连续内存空间
 - block 之间通过 block table 建立映射关系
- 2. 按需分配 (On-demand Allocation)**
 - 仅在 token 真正生成时才分配新 block
 - 彻底消除内部碎片（不预分配）
 - 请求实际长度 ≠ 预分配长度，差值为碎片
- 3. 等大小块 (Uniform Block Size)**
 - 所有 block 大小相同
 - 消除外部碎片：块大小统一，不存在内存空洞
 - 简化内存管理：类似 buddy allocator 的 freelist
- 4. 块级共享 (Block-level Sharing)**
 - 同一请求的多个序列可共享 block
 - 不同请求之间也可通过 copy-on-write 共享
 - 核心应用：parallel sampling、beam search、prefix caching

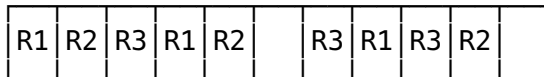
1.3 内存效率对比

现有系统（FasterTransformer / Orca）：

Req1 (2K)	Free (碎)	Req2 (2K)	Free (碎)	Req3 (2K)	连续预分配，碎片丛生 利用率：20-38%



vLLM (PagedAttention):



按需block, 利用率 96%+

block granularity, 无碎片, 支持共享

2. 整体推理流程

2.1 请求生命周期

用户请求到达



```
Scheduler.schedule()  
(vllm/v1/core/sched/scheduler.py:388-1124)  
737 行, 82 圈复杂度, 47 个 callee
```

- ① 收集 waiting queue 中的新请求
- ② 计算 token budget (基于显存/block 可用数)
- ③ 决定 prefill 优先还是 decode 优先
- ④ 分配 block → 调用 KVCacheManager
- ⑤ 如果显存不足 → `_preempt_request()` 抢占
- ⑥ 生成 SchedulerOutput (调度决策)



```
Block-level Memory Manager  
(vllm/v1/core/kv_cache_manager.py)
```

```
KVCacheManager.get_block_ids(request_id)  
→ 返回: (block_ids, ...)  
→ 管理: free block pool / allocated / cached
```

```
SingleTypeKVCacheManager  
→ reachable_block_mask(): 哪个 block 可命中  
→ 支持 SWA (Sliding Window Attention) 稀疏
```



```
Worker / Model Runner
```

```
execute_model()  
→ 对每个 layer:  
    ① 从 block table 解析物理 KV cache 地址  
    ② 调用 PagedAttention kernel
```

- ③ 计算 attention scores
- ④ 将新 token 的 KV 写入对应 block



PagedAttention Kernel (GPU)
(csrc/libtorch_stable/attention/)

V1: Grid(num_heads, num_seqs, 1)
V2: Grid(num_heads, num_seqs, partitions)

输入: query, key_cache, value_cache,
 block_tables, seq_lens, block_size
输出: output tensor

2.2 Scheduler.schedule() 核心逻辑

schedule(self, throttle_prefills=False) → SchedulerOutput

- 1. 收集待调度请求
 - scheduled_new_reqs: waiting 队列中的新请求
 - scheduled_resumed_reqs: 被抢占后恢复的请求
 - scheduled_running_reqs: 上一轮未完成的 running 请求
 - 2. 计算 token_budget
 - max_num_scheduled_tokens = f(available_blocks, block_size)
 - prefill_capacity_bound = g(encoder_compute_budget)
 - 3. Prefill 阶段 (首次推理)
 - 分配 num_prompt_tokens 个 block
 - 调用 KVCacheManager.take_new_block_ids()
 - 设置 scheduled_spec_decode_tokens
 - 4. Decode 阶段 (逐 token 生成)
 - 每个 running request 分配 1 个新 block
 - reuse cached blocks (prefix caching hit)
 - 检查 token_budget 剩余
 - 5. 抢占处理
 - 如果 token_budget 耗尽 → _preempt_request()
 - 将 request 从 RUNNING 移回 WAITING
 - 释放已分配 block: _free_request_blocks()
 - 记录 preemption 事件到 log_stats
 - 6. 返回 SchedulerOutput
 - scheduled_new_reqs
 - scheduled_running_reqs
 - preempted_reqs
 - req_to_new_blocks: dict[str, KVCacheBlocks]
 - scheduled_encoder_inputs
-

3. PagedAttention 核心算法

3.1 算法描述

伪代码 - PagedAttention 核心逻辑

```
def paged_attention(query, key_cache, value_cache, block_tables,
                    seq_lens):
    """
    query:          [num_seqs, num_heads, head_size]      # 当前 query
    key_cache:       [num_blocks, num_kv_heads, head_size/x,
                      block_size, x] # KV 缓存池
    value_cache:     [num_blocks, num_kv_heads, head_size, block_size]
    block_tables:    [num_seqs, max_num_blocks_per_seq]    # 页表
    seq_lens:        [num_seqs]                            # 每请求实际
                      长度
    """
    for seq_id in range(num_seqs):                          # 对每个请求
        num_blocks = ceil(seq_lens[seq_id] / block_size)

        for head_id in range(num_heads):                    # 对每个
            attention head
            output[head_id] = 0

            for block_idx in range(num_blocks):              # 遍历该请求
                的所有 block
                physical_block = block_tables[seq_id][block_idx]
                # 页表映射

                # 从 KV cache 池读取
                k_block = key_cache[physical_block]          #
                [num_kv_heads, ...]
                v_block = value_cache[physical_block]

            # 计算 attention:  $Q @ K^T / \sqrt{d_k}$ , 然后 softmax @ V
            scores = query[head_id] @ k_block[head_id].T /
            sqrt(head_size)

            # 处理 block 内的部分 token
            if block_idx == num_blocks - 1:
                # 最后一个 block 可能不满
                valid_tokens = seq_lens[seq_id] % block_size
                scores = scores[:, :valid_tokens]

            attn_weights = softmax(scores)
            output[head_id] += attn_weights @ v_block[head_id]

    return output
```

3.2 block_tables 数据结构

```
block_tables = [
    [3, 7, 12, -1, -1], # Seq 0: 逻辑 block 0→物理 3, 1→7, 2→12
    [1, 5, 8, 15, -1], # Seq 1: 使用 block 1,5,8,15
    [2, 9, -1, -1, -1], # Seq 2: 使用 block 2,9
]
# -1 表示该槽位未使用 (类似 OS 中页表项的 "invalid" 位)
# max_num_blocks_per_seq = 5 (预分配 5 个逻辑槽位)
```

类比 OS 页表：

```
虚拟地址 (逻辑 block id) → 物理地址 (物理 block id)
Seq0.block[0]           → 物理 block #3
Seq0.block[1]           → 物理 block #7
Seq0.block[2]           → 物理 block #12
```

3.3 V1 vs V2 Kernel

维度	PagedAttention V1	PagedAttention V2
源码文件	paged_attention_v1.cu	paged_attention_v2.cu
Grid 维度	(num_heads, num_seqs, 1)	(num_heads, num_seqs, max_num_partitions)
分区策略	单分区, 直接输出	多分区并行, 再 reduce
灵感来源	FasterTransformer MHA	FlashDecoding (partition 思想)
输出方式	直接写 output tensor	先写 tmp_out → reduce kernel → output
核心 kernel 行数	29 行 (wrapper)	411 行 (paged_attention_kernel)
适用场景	短序列	长序列 + 高并发
ROCm 变体	-	3 个变体 (mfma4/mfma16/reduce, 498 行)

V2 的工作流程：

Step 1: paged_attention_v2_kernel (多分区并行)
Grid: (num_heads, num_seqs, max_num_partitions)
每个 thread block 计算一个 (head, seq, partition) 的结果
→ 写入 tmp_out

Step 2: paged_attention_v2_reduce_kernel

Grid: (num_heads, num_seqs)
将 tmp_out 中多个 partition 的结果归约为最终 output
→ 写入 output

Step 3: 清理临时张量 (tmp_out, exp_sums, max_logits)

3.4 复杂度最高的 Kernel 函数

函数	行数	圈复杂度	循环深度	关键特征
paged_attention_kernel	411	40	3	V2 核心, 22 参数
paged_attention_ll4mi_QKV_mfma4_kernel	498	55	3	ROCM 最大变体, 20 参数
paged_attention_ll4mi_QKV_mfma16_kernel	401	35	3	ROCM mfma16, 20 参数
paged_attention_ll4mi_reduce_kernel	197	27	2	Reduce, 8 参数
test_paged_attention	282	10	4	测试函数, 传递深度 4

4. Block 内存管理机制

4.1 核心数据结构

GPU 显存

Block Pool (固定大小的 block 数组)
Block[0]: [num_kv_heads, head_size, block_size]
Block[1]: [num_kv_heads, head_size, block_size]
Block[2]: FREE
...
Block[N]: [num_kv_heads, head_size, block_size]

Free Block List: [2, 5, 11, 18, ...] (可用 block 编号)

Allocated Block Map: {request_id → [block_ids]}
Cached Block Map: {hash → block_id} (prefix cache)

4.2 KVCacheManager 关键操作

vllm/v1/core/kv_cache_manager.py

```
class KVCacheManager:
    def get_block_ids(request_id: str) → tuple[list[int], ...]:
        """获取分配给指定请求的所有 block ID"""
        # 返回 allocated block_ids

    def take_new_block_ids() → list[int]:
        """从 free pool 分配新 block"""
        # 从 freelist 取, 如果不够 → 触发抢占

    def _free_block(block_id: int):
        """释放单个 block 回 free pool"""
        # 加入 freelist, 同时清理 cache entry

class KVCacheBlocks:
    def get_block_ids() → list[int]:
        """返回此请求的 block ID 列表"""

    def get_unhashed_block_ids() → list[int]:
        """返回未被 hash 缓存的 block (需要进行 hash 计算)"""
```

4.3 Prefix Caching 机制

请求 A: "What is the capital of France?"

Block 0: "What is the " → hash=0x3A7F

Block 1: "capital of " → hash=0xB2E1

Block 2: "France?" → hash=0x8C4D

请求 B: "What is the capital of Germany?"

Block 0: "What is the " → hash=0x3A7F ← 命中! 复用 Block 0

Block 1: "capital of " → hash=0xB2E1 ← 命中! 复用 Block 1

Block 2: "Germany?" → hash=0xD5F2 ← 新分配

请求 B 只需分配 1 个新 block (而不是 3 个)

Block Hash 计算: 对 block 内的 token 序列计算 hash, 存入 cached block map。**命中判定:** 新请求的 block hash 匹配缓存中的 hash → 直接复用物理 block。**LRU 淘汰:** 缓存满时淘汰最久未使用的 block。

4.4 抢占 (Preemption)

当 GPU 显存不足时，Scheduler 触发抢占：

```
_preempt_request(request, timestamp):  
    ① request.status: RUNNING → PREEMPTED  
    ② _free_request_blocks(request) # 释放占用的所有 block  
    ③ 将 request 移回 waiting queue (prepend, 优先恢复)  
    ④ 记录 log_stats 和 preemption 事件
```

两种抢占策略（论文提出的）：- **Swapping**: 将 block 换出到 CPU 内存，恢复时换回（低延迟）- **Recomputation**: 直接丢弃 block，恢复时重新计算（省内存）

5. 调度器设计

5.1 Scheduler 核心数据结构

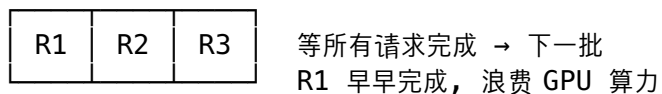
Scheduler

- waiting queue: 新到达的请求（FIFO）
- running queue: 正在执行的请求
- preempted queue: 被抢占的请求（优先恢复）
- token_budget: 本轮可调度的最大 token 数
- kv_cache_manager: Block 内存管理器引用

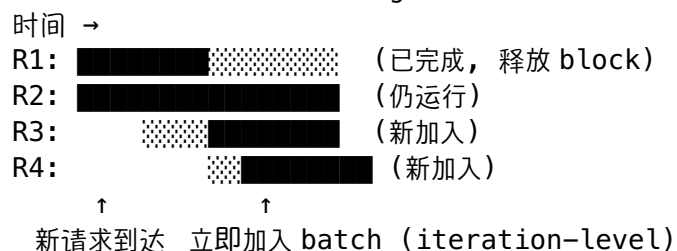
5.2 Continuous Batching

与 PagedAttention 协同设计的关键优化：

传统 Static Batching:



vLLM Continuous Batching:



5.3 schedule() 函数剖析

schedule() - 737 行，82 圈复杂度，7 层最大传递深度

参数: throttle_prefills (是否限制 prefill)
返回: SchedulerOutput

内部流程:

- ① `_pause_state` 检查 → `PAUSED_ALL` 时跳过
- ② 计算 `token_budget = f(kv_cache_manager.available_blocks)`
- ③ 遍历 `waiting queue` → `scheduled_new_reqs`
- ④ 遍历 `running queue` → `scheduled_running_reqs`
- ⑤ `prefill` 优先? 还是 `decode` 优先?
 - └ `defer_prefills = True` 时优先 `decode`
 - └ 否则优先 `prefill` (减少 `first-token latency`)
- ⑥ 对每个请求分配 `block` → `req_to_new_blocks`
- ⑦ 如果 `token_budget` 不足 → `_preempt_request`
- ⑧ 返回 `SchedulerOutput (dict of lists)`

6. 代码实现分析

6.1 代码分布

vLLM 代码库: 83,139 nodes / 490,393 edges

`vllm/v1/core/sched/scheduler.py` ← 调度器 (737行, 82复杂度)
`vllm/v1/core/kv_cache_manager.py` ← Block 管理器
`vllm/v1/core/single_type_kv_cache_manager.py` ← 单类型 KV cache
`vllm/v1/attention/ops/paged_attn.py` ← PagedAttention Python 接口
`csrc/libtorch_stable/attention/` ← CUDA kernel 实现
`csrc/rocm/attention.cu` ← AMD ROCm 后端

`tests/kernels/attention/` ← 测试 (6 个测试文件覆盖)

6.2 热点函数 (fan-in 排序, 取前 5)

函数	fan-in	模块	含义
<code>FlatLogprobs.append</code>	1,659	<code>logprobs</code>	被 1659 处调用
<code>StandaloneCompiledArtifacts.get</code>	916	<code>compilation</code>	编译缓存查询
<code>VllmPatternReplacement.empty</code>	819	<code>compiler pass</code>	编译器模式替换
<code>IntermediateTensors.items</code>	770	<code>sequence</code>	中间张量迭代
<code>_SubprocessWrapper.join</code>	720	<code>v1.utils</code>	

函数	fan-in	模块	含义
			子进程 同步等 待

6.3 PagedAttention Kernel 签名

```
// V2 kernel - 最复杂版本
void paged_attention_kernel(
    float* exp_sums,           // [num_seqs, num_heads,
                               max_num_partitions]
    float* max_logits,        // [num_seqs, num_heads,
                               max_num_partitions]
    scalar_t* out,             // output tensor
    const scalar_t* q,         // query [num_seqs, num_heads,
                               head_size]
    const cache_t* k_cache,    // key cache pool
    const cache_t* v_cache,    // value cache pool
    const int num_kv_heads,
    const float scale,         // 1/sqrt(head_size)
    const int* block_tables,   // ★ 核心: 页表
    const int* seq_lens,       // 请求实际长度
    const int max_num_blocks_per_seq,
    const float* alibi_slopes, // ALiBi 位置编码
    const int q_stride,
    const int kv_block_stride,
    const int kv_head_stride,
    const float* k_scale,      // FP8 量化 scale
    const float* v_scale,
    const int tp_rank,         // tensor parallel rank
    const int blocksparse_local_blocks,
    const int blocksparse_vert_stride,
    const int blocksparse_block_size,
    const int blocksparse_head_sliding_step
)
// Grid: (num_heads, num_seqs, max_num_partitions)
// 411 行, 40 圈复杂度, 19 层循环, 3 层嵌套
```

6.4 ROCm (AMD GPU) 适配

vLLM 同时支持 NVIDIA CUDA 和 AMD ROCm, PagedAttention 有完整的 ROCm 实现:

```
csrc/rocm/attention.cu:
├─ paged_attention_rocm()           ← Python 可调用入口 (3
callers)
├─ paged_attention_custom_launcher() ← 标准 launcher (155行, 28复
杂度)
├─ paged_attention_custom_launcher_navi() ← Navi 架构 launcher
(172行, 36复杂度)
```

- └─ paged_attention_ll4mi_QKV_mfma16_kernel() ← MFMA16 变体 (401行, 35复杂度)
- └─ paged_attention_ll4mi_QKV_mfma4_kernel() ← MFMA4 变体 (498行, 55复杂度, 最大)
- └─ paged_attention_ll4mi_reduce_kernel() ← Reduce (197行, 27复杂度)

平台选择函数 use_rocm_custom_paged_attention() (37行) 根据 GPU 型号和参数选择最优 kernel。

7. 关联生态

7.1 项目依赖图



7.2 同类系统对比

系统	核心技术	论文	KV Cache 策略
vLLM	PagedAttention	SOSP 2023	Block 分页管理
SGLang	RadixAttention	NeurIPS 2024	Radix Tree LRU 缓存
FasterTransformer	手写 CUDA kernel	—	连续内存, 预分配
Orca	Iteration-level batching	OSDI 2022	连续内存

7.3 跨资源查询路径

本 KG 中可以通过以下路径查询 PagedAttention 相关信息：

用户提问 "PagedAttention 相关项目"

- ├ by-tag['vllm'] → 6 projects + 1 paper
- ├ by-tag['pagedattention'] → 1 project + 1 paper
- ├ by-tag['推理'] → 2 projects + 2 papers
- └ manifest.references['pagedattention'].related_projects → vllm, vllm-ascend

总结

PagedAttention 的核心贡献在于将操作系统的虚拟内存分页思想引入 LLM 推理的 **KV cache** 管理。通过 block 级别的内存分配、按需分配、统一大小和块级共享四个设计原则，将 KV cache 利用率从 20-38% 提升到 96%+，吞吐量提升 2-4×。

vLLM 的实现体现了工程上的深思熟虑：737 行的调度器在 prefill/decode 之间动态平衡，411 行的 CUDA kernel 在 GPU 上高效执行分页注意力，完整的 block 管理器支持 prefix caching 和抢占式调度。ROCm/CUDA 双平台支持使其成为 LLM 推理的事实标准。

本报告由 *KG System* 自动生成 | 数据来源: *codebase-memory* 深度索引 (83k nodes) + 跨资源查询 (*by-tag/manifest*) + 论文中英双语翻译